

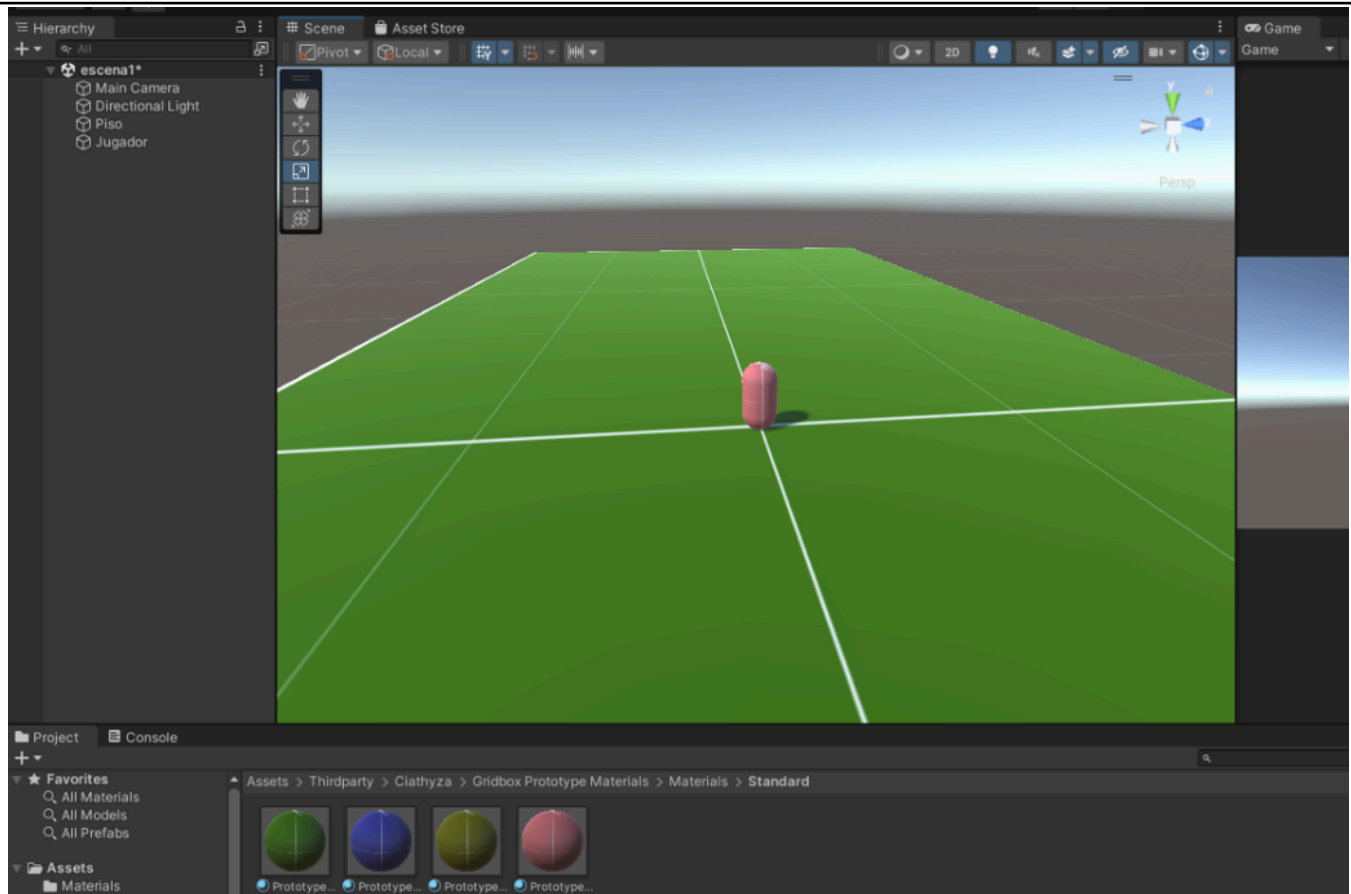
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	<i>CONTROLADORES DE ENTRADA DE PRIMERA Y TERCERA PERSONA</i>				
NÚMERO DE PRÁCTICA:	<i>04</i>	AÑO LECTIVO:	<i>2026A</i>	NRO. SEMESTRE:	<i>IX</i>
FECHA DE PRESENTACIÓN	<i>14/05/2026</i>	HORA DE PRESENTACIÓN	<i>22:00</i>		
INTEGRANTE (s): Vera Zapacayo Jhoer Luis				NOTA:	
DOCENTE(s): <i>Ing. Nieto Valencia Rene Alonso</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Escena 1: Controlador de Personaje y Cámara en tercera persona Creación de Escenario y jugador



Se utilizó el tutorial proporcionado para el laboratorio como guía para implementar el control del personaje:

<https://www.youtube.com/watch?v=7nxpDwnU0uU>

Controlador de personaje en tercera persona

```
CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class ThirdPersonCharacterControl : MonoBehaviour
{
    public float Speed;

    void Update()
    {
        PlayerMovement();
    }

    void PlayerMovement()
```

```
{  
    // Obteniendo ejes de coordenadas verticales y horizontales.  
    float hor = Input.GetAxis("Horizontal");  
    float ver = Input.GetAxis("Vertical");  
  
    // Vector de movimiento  
    Vector3 playerMovement = new Vector3(hor, 0f, ver) * Speed * Time.deltaTime;  
  
    // Aplicando transform para que el personaje se mueva según el vector  
    transform.Translate(playerMovement, Space.Self);  
}
```

La lógica de control de la cámara en tercera persona se desarrolló también con el tutorial sugerido en el laboratorio.

Control de cámara en tercera persona

```
CSharp  
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
public class ThirdPersonCameraControl : MonoBehaviour  
{  
    // Velocidad de rotación de la cámara  
    float rotationSpeed = 1;  
  
    // Transforms para el jugador y Target, que es el objetivo de la cámara  
    public Transform Target, Player;  
  
    // Valores de los ejes del mouse  
    float mouseX, mouseY;  
  
    void Start()  
    {  
        // Remover el cursor  
        Obstruction = Target;  
        Cursor.visible = false;  
        Cursor.lockState = CursorLockMode.Locked;  
    }  
  
    private void LateUpdate()  
    {  
        CamControl();  
    }  
  
    void CamControl()
```

```
{  
    // Control de la cámara con el mouse  
    mouseX += Input.GetAxis("Mouse X") * rotationSpeed;  
    mouseY -= Input.GetAxis("Mouse Y") * rotationSpeed;  
  
    // Límites para que la cámara no se voltee  
    mouseY = Mathf.Clamp(mouseY, -35, 60);  
  
    // La cámara se fija en un objetivo  
    transform.LookAt(Target);  
  
    // Controlar la rotación de la cámara con el mouse y la rotación del personaje  
    Target.rotation = Quaternion.Euler(mouseY, mouseX, 0);  
    Player.rotation = Quaternion.Euler(0, mouseX, 0);  
}
```

Para implementar el control de colisiones de la cámara y solucionar problemas de visibilidad, se usó el tutorial del mismo autor disponible en:

<https://www.youtube.com/watch?v=wWyx7-clxP8>

Con el fin de evitar que la cámara pierda visibilidad del objetivo cuando un objeto se interpone entre ambos, se aplicará una modificación al script de control de cámara propuesta en dicho tutorial.

Control de cámara en tercera persona con detección de obstrucciones

```
CSharp  
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
public class ThirdPersonCameraControl : MonoBehaviour  
{  
    // Velocidad de rotación de la cámara  
    float rotationSpeed = 1;  
  
    // Transforms para el jugador y Target, que es el objetivo de la cámara  
    public Transform Target, Player;  
  
    // Valores de los ejes del mouse  
    float mouseX, mouseY;  
  
    // Transform para la obstrucción  
    public Transform Obstruction;  
  
    // Velocidad del zoom  
    float zoomSpeed = 2f;
```

```
void Start()
{
    // Valor inicial para transform Obstruction
    Obstruction = Target;

    Cursor.visible = false;
    Cursor.lockState = CursorLockMode.Locked;
}

private void LateUpdate()
{
    CamControl();
    ViewObstructed();
}

void CamControl()
{
    // Control de la cámara con el mouse
    mouseX += Input.GetAxis("Mouse X") * rotationSpeed;
    mouseY -= Input.GetAxis("Mouse Y") * rotationSpeed;

    // Límites para que la cámara no se voltee
    mouseY = Mathf.Clamp(mouseY, -35, 60);

    // La cámara se fija en un objetivo
    transform.LookAt(Target);

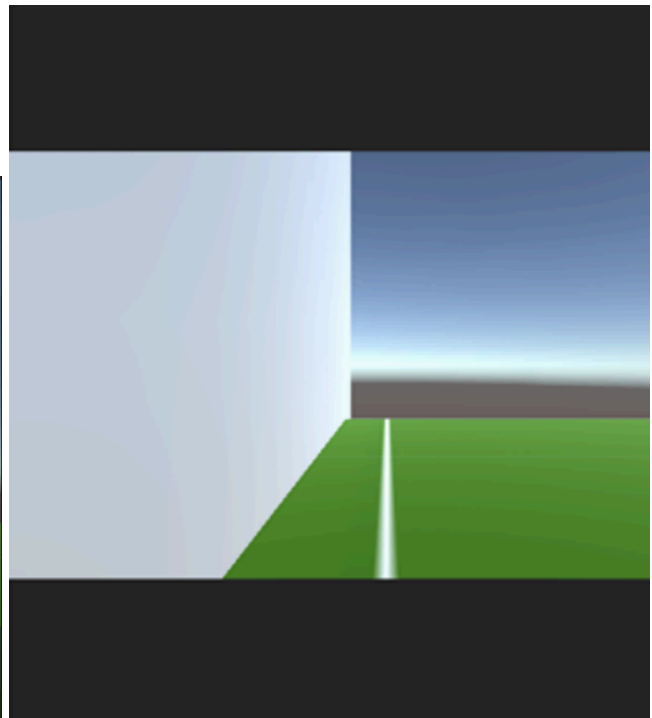
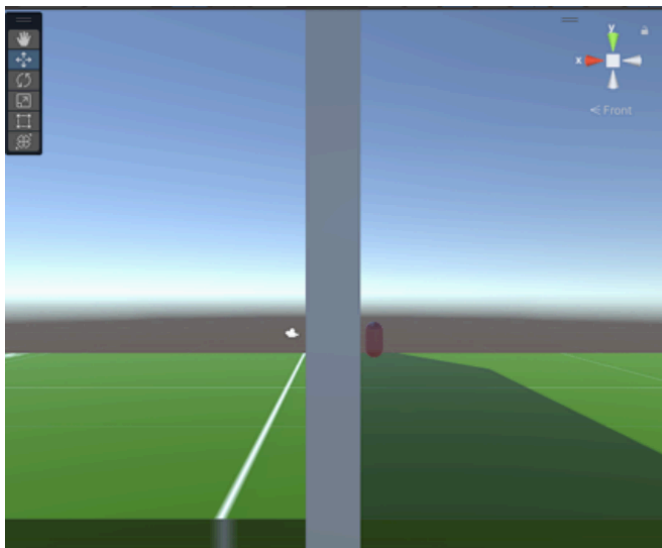
    // Control para permitir que solo se mueva la cámara cuando se presiona la tecla
Shift
    if (Input.GetKey(KeyCode.LeftShift))
    {
        Target.rotation = Quaternion.Euler(mouseY, mouseX, 0);
    }
    else
    {
        // Controlar la rotación de la cámara con el mouse y la rotación del
personaje
        Target.rotation = Quaternion.Euler(mouseY, mouseX, 0);
        Player.rotation = Quaternion.Euler(0, mouseX, 0);
    }
}

// Método para detectar obstrucciones
void ViewObstructed()
{
    // Se hace uso de Raycast para detectar si un objeto está en la línea de visión
    RaycastHit hit;
```

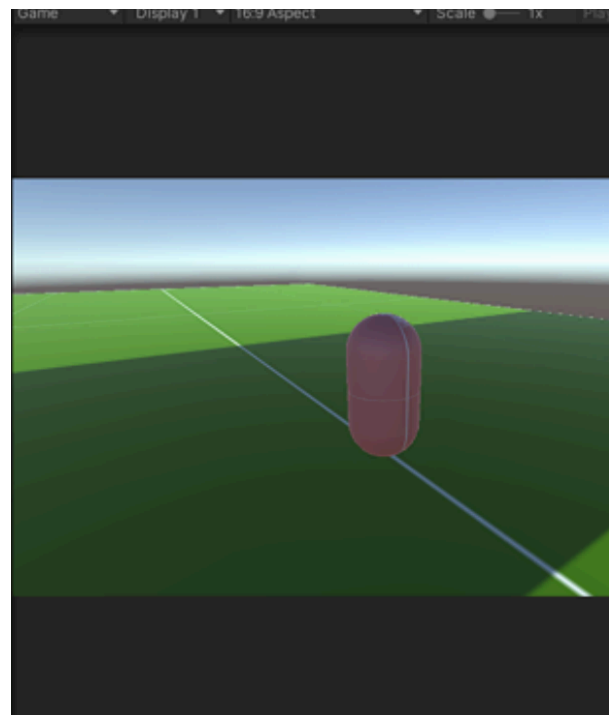
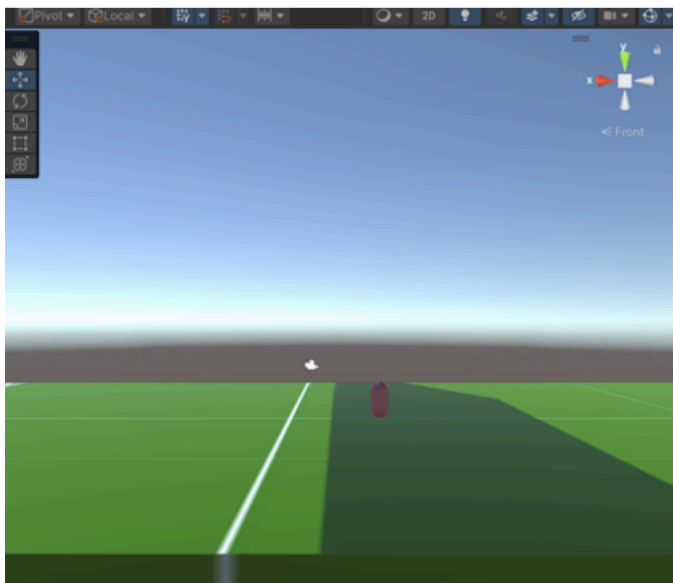
```
if (Physics.Raycast(transform.position, Target.position - transform.position, out  
hit, 4.5f))  
{  
    // Si el objeto que está en la línea de visión no es el jugador  
    if (hit.collider.gameObject.tag != "Player")  
    {  
        Obstruction = hit.transform;  
  
        // Usar shadow casting para ocultar la obstrucción pero mantener las  
sombas  
        Obstruction.gameObject.GetComponent<MeshRenderer>().shadowCastingMode =  
            UnityEngine.Rendering.ShadowCastingMode.ShadowsOnly;  
  
        // Verificar distancia de la cámara y el jugador para decidir si  
acercarse o no  
        if (Vector3.Distance(Obstruction.position, transform.position) >= 3f &&  
            Vector3.Distance(transform.position, Target.position) >= 1.5f)  
        {  
            // Zoom  
            transform.Translate(Vector3.forward * zoomSpeed * Time.deltaTime);  
        }  
    }  
    else  
    {  
        // Activar el MeshRenderer del muro para que vuelva a ser visible  
        Obstruction.gameObject.GetComponent<MeshRenderer>().shadowCastingMode =  
            UnityEngine.Rendering.ShadowCastingMode.On;  
  
        if (Vector3.Distance(transform.position, Target.position) < 4.5f)  
        {  
            transform.Translate(Vector3.back * zoomSpeed * Time.deltaTime);  
        }  
    }  
}  
}
```

La solución implementada para los problemas de visibilidad consiste en utilizar ray tracing para verificar si el objeto frente a la cámara corresponde al jugador. Si no lo es, se emplea shadow casting para ocultarlo visualmente sin eliminar sus sombras, generando así la apariencia de que la cámara atraviesa los objetos sin bloquear la vista del objetivo.

En este caso, la presencia de una pared entre la cámara y el objetivo bloquea completamente la vista del personaje.

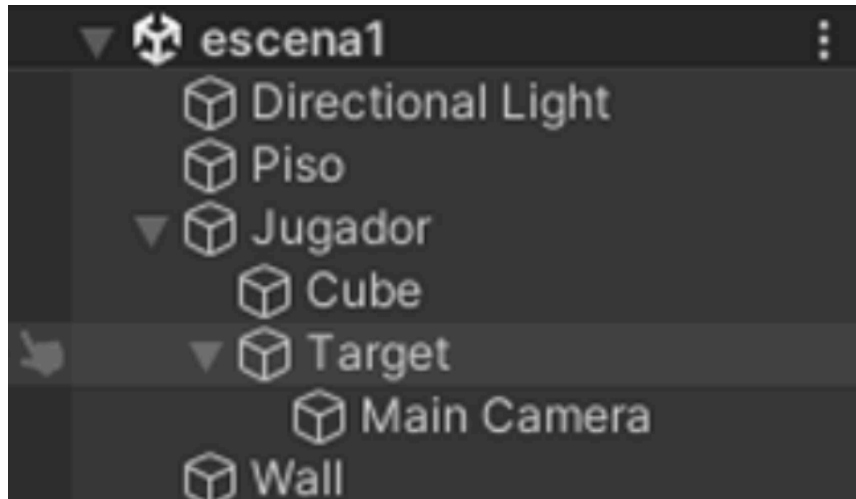


La imagen muestra el resultado tras aplicar la técnica que oculta el muro entre la cámara y el objetivo.



	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

Como se puede observar, el muro desaparece visualmente, pero sus sombras siguen presentes. De esta manera, el objetivo permanece visible para la cámara.

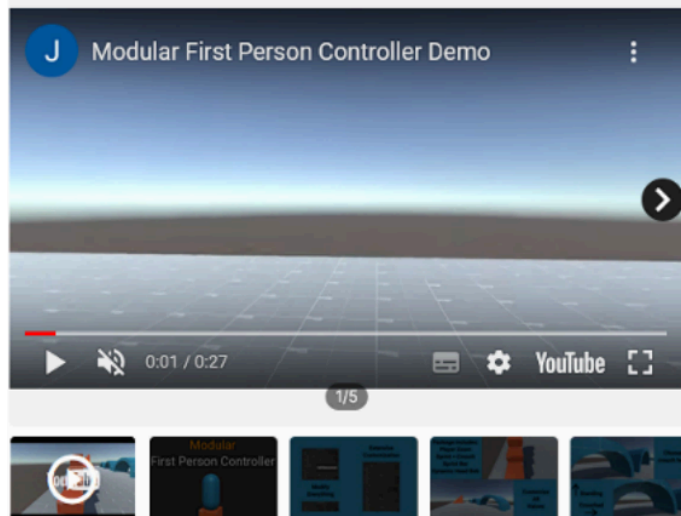


Escena 2: Uso de diferentes tipos de cámaras.

En esta escena se emplearán tres tipos de cámara:

- **Cámara 1:** Primera persona
- **Cámara 2:** Cámara fija en una zona específica de la escena
- **Cámara 3:** Cámara cenital

Para el control del movimiento del personaje y la cámara en primera persona, se utilizará el siguiente módulo disponible en la tienda de Unity:



Modular First Person Controller

JeCase ★★★★★ (202) | ❤️ (5523)

FREE
Taxes/VAT calculated at checkout

Product Information ^

License agreement [Standard Unity Asset Store EULA](#)

File size 366.2 KB

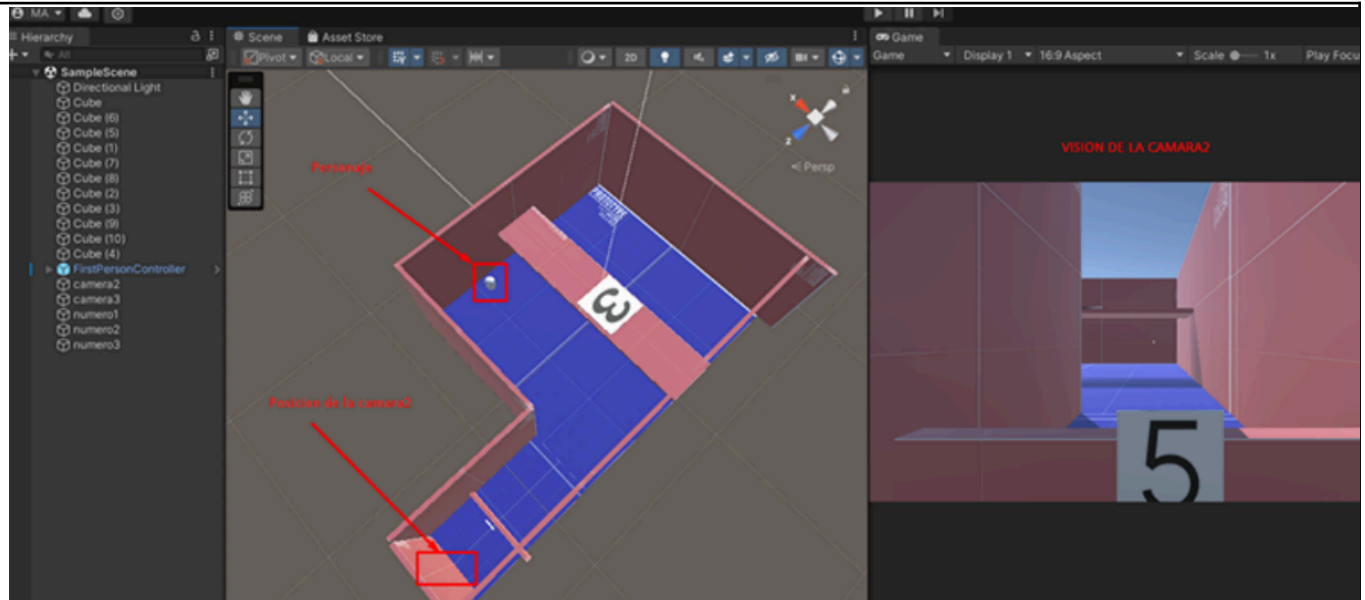
Latest version 1.0.1

Latest release date Apr 12, 2021

Original Unity version 2019.4.20 or higher

Reviews ★★★★★

Este módulo nos permitirá controlar el movimiento de nuestro personaje y además utilizar una cámara en primera persona. Para la cámara 2 se emplea una cámara fija, mientras que la cámara 3 será una cámara cenital que ofrecerá una vista completa de la escena.



La jugabilidad en esta escena consiste en que el usuario debe encontrar los cuatro números ocultos dentro del escenario. Para lograrlo, tendrá que utilizar los distintos tipos de cámaras disponibles para poder localizarlos.

Script para el cambio de cámaras:

Cambio entre múltiples cámaras

```
CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class CameraSwitcher : MonoBehaviour
{
    public Camera camera1;
    public Camera camera2;
    public Camera camera3;

    void Start()
    {
        ActivateCamera(camera1);
    }

    void Update()
    {
        Debug.Log("Update está funcionando");

        if (Input.GetKeyDown(KeyCode.T))
        {

```

```
        ActivateCamera(camera1);
        Debug.Log("Camara 1 activa");
    }
    else if (Input.GetKeyDown(KeyCode.Y))
    {
        ActivateCamera(camera2);
        Debug.Log("Camara 2 activa");
    }
    else if (Input.GetKeyDown(KeyCode.U))
    {
        ActivateCamera(camera3);
        Debug.Log("Camara 3 activa");
    }
}

void ActivateCamera(Camera camToActivate)
{
    camera1.gameObject.SetActive(false);
    camera2.gameObject.SetActive(false);
    camera3.gameObject.SetActive(false);

    camToActivate.gameObject.SetActive(true);
}
}
```

Se utiliza el método ActivateCamera para activar la cámara correspondiente según el botón que el usuario presione.

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Qué problemas pueden presentar las cámaras de tercera persona?

El principal problema que puede presentarse es la pérdida de visibilidad del objetivo, ya sea por una interrupción en la línea de visión o porque la cámara no cuenta con el espacio suficiente para maniobrar, lo que termina afectando su ángulo de visión. Otros inconvenientes pueden incluir una mala configuración en el movimiento de la cámara, lo cual puede provocar desorientación en el jugador.

2. ¿Cómo han realizado el control de múltiples cámaras en la escena?

Se utilizaron botones individuales para cada cámara. Se definió una cámara inicial y, para cambiar entre cámaras, se empleó el método `camToActivate.gameObject.SetActive(true)` con el fin de activar la cámara seleccionada.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

III. CONCLUSIONES

- Las cámaras son el principal medio mediante el cual el usuario interactúa y recibe información del juego, por lo tanto, tienen un impacto directo en la jugabilidad.
- El tipo de cámara influye en el nivel de inmersión que se puede generar durante la experiencia del jugador.
- Para implementar una cámara que siga al jugador, es necesario asignar un objetivo (target), que puede ser, por ejemplo, el hombro del personaje.
- Una solución a los problemas de colisión y visibilidad de la cámara consiste en aplicar shadow casting, lo que permite ocultar la malla de un objeto que obstruye la vista sin eliminar sus propiedades físicas ni sus sombras.
- El uso de múltiples cámaras dentro de una misma escena brinda variedad y dinamismo a la jugabilidad, permitiendo diferentes perspectivas y modos de exploración.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1]<https://www.youtube.com/watch?v=7nxdWnU0uU>
- [2]https://www.youtube.com/watch?v=wWyx7_clxP8 Controlador de Usuario para la escena 2
- [3] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*. Stylus Publishing, LLC.
- [4] D. P. Kothari, G. Awari, D. Shrimankar, A. Bhende (2019), *Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction*